

PHASE 1 — PROJECT PROPOSAL

CryptoLens

A Medallion Architecture Pipeline for Binance Vision Kline Data

Course: Data Analysis and Visualization — Semester Project

Team: Arqum Umais(24L-2591) & Abdullah Saeed(24L-2529)

GitHub Repository: <https://github.com/x-abdullah-saeed-x/CryptoLens>

Submission Date: 26 September 2026

Platform: Databricks Community Edition

Project Overview & Objective

This project builds an end-to-end, automated Lakehouse pipeline on Apache Spark that ingests real-world cryptocurrency market data from Binance Vision, processes it through the Bronze, Silver, and Gold layers of the Medallion Architecture, and culminates in a Power BI dashboard for market analysts. The domain is financial markets / crypto trading analytics: specifically, historical OHLCV (Open, High, Low, Close, Volume) candlestick data for a curated set of USDT trading pairs on Binance's spot market.

The pipeline will support both a Full Load (a multi-month historical backfill) and an Incremental Load (new daily files appended as they are published), giving the team a realistic base to practice batch ingestion, Delta Lake MERGE semantics, data quality enforcement, and dimensional modeling — while staying well within the compute and storage limits of a free-tier Spark environment.

1. Domain & Source Identification

1.1 Project Concept

Domain: Financial Markets / Cryptocurrency Trading Analytics. The pipeline tracks price action and trading activity across a fixed basket of Binance spot trading pairs (e.g., BTCUSDT, ETHUSDT, BNBUSDT, SOLUSDT, XRPUSDT) so that, downstream, analysts can study volatility, trend, and co-movement between assets over time — the kind of analysis a trading desk or a research team would want on a daily cadence.

1.2 Data Source

Provider: Binance Vision — Binance's official public historical market data archive.

Base URL: <https://data.binance.vision>

Dataset used: Spot Klines (candlestick data), sourced from the same records as the `/api/v3/klines` REST endpoint.

Exact file path convention:

- Monthly (full load): `/data/spot/monthly/klines/<SYMBOL>/<INTERVAL>/<SYMBOL>-<INTERVAL>-<YYYY>-<MM>.zip`
- Daily (incremental): `/data/spot/daily/klines/<SYMBOL>/<INTERVAL>/<SYMBOL>-<INTERVAL>-<YYYY>-<MM>-<DD>.zip`

Example: <https://data.binance.vision/data/spot/monthly/klines/BTCUSDT/1h/BTCUSDT-1h-2026-08.zip>. Each archive is a zipped, headerless CSV; every file ships with a companion `.CHECKSUM` sidecar (SHA-256) that the ingestion job will verify before processing. The interval chosen for this project is 1-hour (1h) candles, which balances analytical resolution against file/row volume.

1.3 Ingestion Pattern

Full Load — Download the monthly archives for the last 18–24 months for each of the 5 tracked symbols. Klines are immutable once a candle has closed, so a monthly archive never changes retroactively, making this a clean, append-only historical baseline with no reconciliation needed.

Incremental Load — Each day, Binance publishes the previous UTC day's data as a new daily archive per symbol/interval. The pipeline will poll for and download the newest daily file per symbol on a schedule (e.g., a Databricks Job run once every 24 hours), verify its checksum, and append it to Bronze. Because candles are

immutable, the incremental step is a straightforward append rather than an upsert/delete — this is called out explicitly in Section 4 (Silver modeling) since it is simpler than the update/delete patterns used by, say, Open Food Facts or Inside Airbnb.

2. Data Samples & Volume Estimation

2.1 Sample Files Provided

Two representative raw payloads are included in the /samples folder of the GitHub repository (see Section 6):

- BTCUSDT-1h-2026-08-SAMPLE.csv — a Full Load-style payload: 96 hourly candles (4 representative days) drawn from what a ~720-row August 2026 monthly file would contain.
- BTCUSDT-1h-2026-09-25.csv — an Incremental Load payload: 24 hourly candles, i.e. one complete daily file exactly as Binance publishes it.

Note: our sandboxed development environment does not currently have outbound network access to data.binance.vision, so these two sample files were synthetically generated to exactly match Binance's real column order, data types, and realistic value ranges (verified against Binance's published schema and the binance/binance-public-data reference repository). Before Phase 2, the team will download and checksum-verify the actual production files directly from Binance Vision to replace these placeholders — the ingestion code itself is unaffected, since the schema is identical.

Both files share the same 12-column, headerless layout:

#	Column	Type	Example value
1	Open time	epoch ms	1785542400000
2	Open	decimal string	112500.00
3	High	decimal string	112754.43
4	Low	decimal string	112335.46
5	Close	decimal string	112688.23
6	Volume (base asset)	decimal string	317.40805361
7	Close time	epoch ms	1785545999999
8	Quote asset volume	decimal string	35738278.28
9	Number of trades	integer	1639
10	Taker buy base volume	decimal string	169.92119488
11	Taker buy quote volume	decimal string	19132126.23
12	Ignore	constant	0

2.2 Volume & Frequency Estimation

Scope: 5 symbols (BTCUSDT, ETHUSDT, BNBUSDT, SOLUSDT, XRPUSDT) × 1h interval × 24 months of history.

Load type	Cadence	Files	Rows (approx.)	Size (approx.)
Full load (Bronze)	One-time backfill	5 symbols × 24 monthly files = 120 files	~86,400 rows (720 rows/file)	~6–9 MB zipped, ~25–35 MB unzipped CSV
Incremental load	Daily, per symbol	5 files/day (1 per symbol)	~120 rows/day (24 rows/file)	~15–25 KB/day zipped
Incremental (weekly)	Rolled up	35 files/week	~840 rows/week	~100–175 KB/week
Silver table (24 mo. + growth)	Cumulative	n/a — Delta table	~87,000+ rows, growing ~120/day	~10–15 MB as Delta/Parquet

This is intentionally a small, free-tier-friendly footprint (single-digit MB to low tens of MB across the full project lifetime), while still exercising real Spark partitioning, Delta MERGE-adjacent patterns, and dimensional modeling at meaningful (though not "big data"-scale) row counts. Symbol/date range can be widened later at negligible extra cost if more volume is desired for the Phase 2/3 demo.

3. Security & Compliance

3.1 PII / Sensitive Data Assessment

The dataset contains no Personally Identifiable Information. Every field in a kline record — open/close time, OHLC prices, volume, quote volume, trade count, taker-buy volumes — is an aggregated, anonymous statistic describing market-wide trading activity for a symbol over a fixed time bucket. There are no usernames, account IDs, order IDs, wallet addresses, IP addresses, emails, or any other identifier tied to an individual trader or counterparty. Binance deliberately publishes this feed as fully aggregated market data for public research and algorithmic-trading use, with no trader-level granularity.

3.2 Handling Strategy

Because no PII is present, no masking, hashing, or column-dropping is required for privacy purposes. The team will still apply standard data-quality and governance controls as the data moves through the Medallion layers:

- Bronze: store raw files immutably (append-only, source-of-truth) with ingestion metadata (source_file, ingestion_timestamp, checksum_verified flag).
- Silver: enforce schema and type constraints (reject rows with malformed prices/volumes or an open_time outside the expected symbol's file period).
- Access control: since the data is already public, no row/column-level security is needed for this dataset — access control in the workspace is limited to standard Databricks workspace permissions for the two team members.

This is stated and justified explicitly here per the assignment's PII-disclosure requirement: this dataset is PII-free by design, and that fact is the deciding factor for a lighter-weight Silver-layer governance approach compared to, e.g., a source with usernames or addresses.

4. High-Level Medallion Data Modeling

4.1 Bronze Layer (Raw)

Bronze stores the data as close to its original form as possible, plus lightweight ingestion metadata:

- Land each downloaded .zip, verify against its .CHECKSUM file, then unzip and land the raw headerless CSV rows unmodified into a Bronze Delta table, partitioned by symbol and ingestion_date.
- Append the following technical columns on ingest: _source_file, _ingestion_ts, _load_type (full | incremental), _checksum_verified.
- No type casting, renaming, or business logic happens in Bronze — it is the immutable audit trail.

4.2 Bronze → Silver: Cleaning & Casting

The Silver layer applies the following transformations:

- Assign proper column names (open_time, open, high, low, close, volume, close_time, quote_asset_volume, num_trades, taker_buy_base_vol, taker_buy_quote_vol) — the source files ship with no header.
- Cast open_time / close_time from epoch milliseconds to a proper UTC timestamp type.
- Cast OHLC price fields and volume fields from string to DECIMAL(18,8) (Binance ships them as fixed-point strings, not native numerics).
- Deduplicate on (symbol, interval, open_time) — the incremental daily file for "today" and next month's monthly file both eventually contain the same candle, so a MERGE / dedup on this natural key prevents double-counting; this is the one place a MERGE-style upsert is used, even though individual candle values themselves never change once closed.
- Derive symbol and interval columns from the source file path/name (they are not embedded in the row data itself).
- Basic quality checks: reject/quarantine rows where high < low, or close outside [low, high], or any null in a required field.

High-level Silver data model — a single cleansed, conformed fact table:

Table	Grain	Key columns
silver.klines	One row per (symbol, interval, open_time)	symbol, interval, open_time, close_time, open, high, low, close, volume, quote_asset_volume, num_trades, taker_buy_base_vol, taker_buy_quote_vol

4.3 Gold Layer

Gold reshapes Silver into business-ready, dimensionally modeled tables for the dashboard:

- dim_symbol — one row per trading pair: symbol, base_asset, quote_asset, first_available_date.
- dim_date — standard calendar dimension (date, day_of_week, week, month, quarter, year) to support time-based slicing.
- fact_daily_ohlc — daily bar rolled up from hourly Silver data: daily open/high/low/close, daily volume, daily trade count, joined to dim_symbol and dim_date.
- gold.volatility_daily — daily % return, rolling 7-day and 30-day realized volatility (stdev of returns), per symbol.
- gold.moving_averages — 7-day and 30-day simple moving average of close price, per symbol, per day, plus a golden/death-cross flag.
- gold.symbol_correlation — a rolling 30-day pairwise Pearson correlation matrix of daily returns across the 5 tracked symbols, refreshed daily.

Together these give analysts pre-aggregated, join-free tables that map directly onto the dashboard visuals in Section 5.

5. Business Intelligence & Dashboards

Tool: Power BI (Desktop), connected to the Gold Delta tables via the Databricks connector.

Business questions the dashboard will answer:

- How has each tracked asset's price and volatility trended over the past N days/weeks?
- Which assets are currently showing a bullish (golden cross) or bearish (death cross) moving-average signal?
- How correlated are these assets right now, and is that correlation rising or falling — useful for portfolio diversification decisions?

Planned visuals:

- Price & volatility trend — a combo line/area chart per symbol: daily close price (line) overlaid with 7-day realized volatility (shaded area), with a symbol slicer and date-range filter.
- Moving-average crossover chart — close price plus 7-day and 30-day SMA lines per symbol, with crossover points highlighted, to flag potential trend reversals.
- Correlation heatmap — a 5×5 matrix visual of the rolling 30-day return correlation across all tracked symbols, updated daily, to spot diversification opportunities or clustering (e.g., alt-coins moving together with BTC).
- (Stretch) KPI cards — 24h volume, 24h % change, and current trade count per symbol, refreshed on each incremental run.

6. Engineering Setup & FinOps

6.1 Version Control

A GitHub repository will host all pipeline code, notebooks, and the two sample data files described in Section 2, structured as:

- /samples — BTCUSDT-1h-2026-08-SAMPLE.csv, BTCUSDT-1h-2026-09-25.csv, README.md (schema documentation)
- /notebooks — Bronze, Silver, and Gold Databricks notebooks (one per layer, or per job step)
- /docs — this proposal and future Phase 2/3 documents, plus an architecture diagram
- /dashboards — exported Power BI (.pbix) file once Gold tables are live

Repository link: <https://github.com/x-abdullah-saeed-x/CryptoLens>

6.2 Cost / FinOps Awareness

Platform: Databricks Community Edition (free), which provides a single small driver-only cluster with auto-terminate — no paid compute is required for this project's data volume.

Given the modest volume estimated in Section 2 (single-digit-to-low-tens of MB across the full 24-month history plus a full semester of daily increments), the team will manage free-tier limits by:

- Developing and unit-testing all notebooks against the small sample files in /samples before pointing them at the full historical backfill, to avoid wasted cluster time on broken code.
- Keeping the symbol list fixed at 5 pairs and the interval fixed at 1h for the semester, rather than scaling to all ~400+ Binance pairs or finer intervals (e.g., 1m), which would multiply file/row counts by 60× or more.
- Using Delta Lake's OPTIMIZE / Z-ORDER sparingly and only on the (small) Gold tables, since Community Edition clusters auto-terminate after inactivity and have limited DBU allowance.
- Scheduling the incremental job to run once daily (not continuously) via a Databricks Job, matching Binance's own daily file publication cadence, to avoid unnecessary cluster spin-ups.
- If the team later needs more headroom (e.g., for Phase 3's dashboard refresh testing), falling back to Azure for Students credits rather than a paid Databricks workspace.